

C++ Programming

Day 2: 조건 분기와 반복 제어

if-else, switch, do-while, break, continue, 난수

Contents

1	학습 목표	2
2	두 가지 경우 선택: if--else	2
2.1	예제: 성인 여부 판별	2
2.2	비교와 대입의 차이	3
3	여러 조건 처리: else if	3
3.1	예제: 학점 판별	3
3.2	조건 검사 순서	4
4	조건문 안의 조건문: Nested if	4
4.1	예제: 단계적 로그인 검사	5
5	하나의 값으로 여러 경우 선택: switch	5
5.1	예제: 메뉴 선택	6
5.2	break와 fall-through	6
5.3	if--else와 switch 비교	7
6	최소 한 번 실행: do--while	7
6.1	while과 do--while 비교	8
7	반복 즉시 종료: break	8
8	현재 반복 건너뛰기: continue	9
8.1	break와 continue 비교	9
8.2	입력값 검증	9
9	난수 생성	10
9.1	rand()	10
9.2	srand()와 seed	10
10	원하는 범위의 난수	11
11	종합 실습: 숫자 맞추기 게임	11
11.1	전체 코드	11
11.2	실행 흐름 분석	12
12	실습 파일 구성	13
13	핵심 요약	13

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- if--else와 else if를 이용한 조건 분기
- 중첩 조건문을 이용한 단계적 판단
- switch문과 break의 역할
- while문과 do--while문의 차이
- break, continue를 이용한 반복 흐름 제어
- rand(), srand(), time()을 이용한 난수 생성
- 조건문, 반복문, 난수를 결합한 숫자 맞추기 게임 구현

Day 1에서는 프로그램의 기본 구조, 입출력, 변수, 연산자, 기본 조건문과 반복문을 학습했다. Day 2에서는 조건문과 반복문을 더 정교하게 사용하여 프로그램의 실행 흐름을 직접 설계한다.

2. 두 가지 경우 선택: if--else

if문은 조건이 참일 때만 코드를 실행한다. 조건이 참일 때와 거짓일 때 서로 다른 작업을 수행하려면 if--else를 사용한다.

```

1 if (condition)
2 {
3     // condition - 128) * 64 + (' - 128) 뽕 true - 128) * 64 + (' - 128) 뽕
4     - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 퀘
5 }
6 else
7 {
8     // condition - 128) * 64 + (' - 128) 뽕 false - 128) * 64 + (' - 128) 뽕
9     - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 퀘
10 }

```

2.1. 예제: 성인 여부 판별

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int age;
8
9     cout << "Age: ";
10    cin >> age;
11
12    if (age >= 20)
13    {
14        cout << "Adult" << endl;
15    }
16    else

```

```

17 {
18     cout << "Minor" << endl;
19 }
20
21 return 0;
22 }

```

입력이 23이면 `age >= 20`이 참이므로 `Adult`가 출력된다. 입력이 17이면 조건이 거짓이므로 `Minor`가 출력된다.

핵심 포인트

`if--else`에서는 두 블록 중 정확히 하나만 실행된다. 서로 동시에 성립할 수 없는 두 경우를 처리할 때 적합하다.

2.2. 비교와 대입의 차이

```

1 number = 10; // - 128) * 64 + (' - 128) 끝 - 128) * 64 + (' - 128) 뽀
2 number == 10; // - 128) * 64 + (' - 128) 끝 - 128) * 64 + (' - 128)

```

`=`는 오른쪽 값을 왼쪽 변수에 저장하는 대입 연산자이고, `==`는 두 값이 같은지 검사하는 비교 연산자이다.

조건식에서 실수로 `number = 10`을 쓰면 10이 변수에 대입되고, 0이 아닌 값이 참으로 평가되어 의도와 다르게 작동할 수 있다.

3. 여러 조건 처리: `else if`

두 가지보다 많은 경우를 처리하려면 `else if`를 사용한다.

```

1 if (condition1)
2 {
3     // condition1 - 128) * 64 + (' - 128) 뽀 true
4 }
5 else if (condition2)
6 {
7     // condition1 - 128) * 64 + (' - 128) 뽀
8     false - 128) * 64 + (' - 128) 뽀 - 128) * 64 + (' - 128)
9     condition2 - 128) * 64 + (' - 128) 끝 true
10 }
11 else
12 {
13     // - 128) * 64 + (' - 128) 뽀 - 128) * 64 + (' - 128) 뽀
14     - 128) * 64 + (' - 128) 끝 - 128) * 64 + (' - 128) 끝
15     - 128) * 64 + (' - 128) 뽀 - 128) * 64 + (' - 128) - 128) * 64 + (' - 128) 뽀
16     false
17 }

```

3.1. 예제: 학점 판별

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int score;

```

```

8
9     cout << "Score: ";
10    cin >> score;
11
12    if (score >= 90)
13    {
14        cout << "Grade A" << endl;
15    }
16    else if (score >= 80)
17    {
18        cout << "Grade B" << endl;
19    }
20    else if (score >= 70)
21    {
22        cout << "Grade C" << endl;
23    }
24    else if (score >= 60)
25    {
26        cout << "Grade D" << endl;
27    }
28    else
29    {
30        cout << "Grade F" << endl;
31    }
32
33    return 0;
34 }

```

3.2. 조건 검사 순서

`else if` 구조에서는 조건을 위에서부터 차례로 검사한다. 처음으로 참이 된 블록 하나만 실행되고, 아래 조건은 더 이상 검사하지 않는다.

```

1 if (score >= 60)
2 {
3     cout << "Grade D" << endl;
4 }
5 else if (score >= 90)
6 {
7     cout << "Grade A" << endl;
8 }

```

점수가 95여도 첫 번째 조건이 이미 참이므로 Grade D가 출력된다. 범위가 겹치는 조건은 더 제한적인 조건부터 작성해야 한다.

핵심 포인트

`else if`는 위에서부터 검사하고 처음 참인 블록 하나만 실행한다. 따라서 조건의 순서가 결과를 바꿀 수 있다.

4. 조건문 안의 조건문: Nested if

조건문 내부에 또 다른 조건문을 작성하는 것을 중첩 조건문이라고 한다.

```

1 if (condition1)
2 {
3     if (condition2)
4     {

```

```

5      // - 128) * 64 + (' - 128) 꿀
      - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) - 128) * 64 + (' - 128) 뽕
      - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 꿀
      - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 뽕
      - 128) * 64 + (' - 128) 꿀
      - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 켜
6  }
7  }

```

4.1. 예제: 단계적 로그인 검사

```

1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main()
7  {
8      string password;
9      bool isAdmin = true;
10
11     cout << "Password: ";
12     cin >> password;
13
14     if (password == "cpp123")
15     {
16         cout << "Login successful." << endl;
17
18         if (isAdmin)
19         {
20             cout << "Administrator mode." << endl;
21         }
22     }
23     else
24     {
25         cout << "Login failed." << endl;
26     }
27
28     return 0;
29 }

```

안쪽의 if (isAdmin)은 바깥쪽 비밀번호 검사가 성공했을 때만 평가된다. 두 조건을 항상 동시에 검사해도 된다면 논리 연산자 &&를 사용할 수 있다.

```

1  if (password == "cpp123" && isAdmin)
2  {
3      cout << "Administrator mode." << endl;
4  }

```

단계별로 다른 메시지나 작업이 필요하다면 중첩 조건문이 자연스럽다. 다만 중첩이 지나치게 깊으면 코드 흐름을 이해하기 어려워진다.

5. 하나의 값으로 여러 경우 선택: switch

하나의 변수나 표현식이 특정 값과 일치하는지 여러 번 비교할 때 switch문을 사용할 수 있다.

```

1  switch (expression)
2  {

```

```

3  case value1:
4      // expression == value1
5      break;
6
7  case value2:
8      // expression == value2
9      break;
10
11 default:
12     //
13     - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 켜 - 128) * 6
14     case - 128) * 64 + (' - 128)
15     - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 뽕
16     - 128) * 64 + (' - 128) 꿀
17     break;
18 }

```

5.1. 예제: 메뉴 선택

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int menu;
8
9      cout << "1. Start" << endl;
10     cout << "2. Settings" << endl;
11     cout << "3. Exit" << endl;
12     cout << "Select: ";
13     cin >> menu;
14
15     switch (menu)
16     {
17         case 1:
18             cout << "Game started." << endl;
19             break;
20
21         case 2:
22             cout << "Settings opened." << endl;
23             break;
24
25         case 3:
26             cout << "Program ended." << endl;
27             break;
28
29         default:
30             cout << "Invalid menu." << endl;
31             break;
32     }
33
34     return 0;
35 }

```

5.2. break와 fall-through

break는 현재 switch문을 즉시 종료한다. break가 없으면 일치한 case 아래의 코드가 계속 실행된다.

```

1 int number = 1;
2
3 switch (number)
4 {
5     case 1:
6         cout << "One" << endl;
7     case 2:
8         cout << "Two" << endl;
9     case 3:
10        cout << "Three" << endl;
11 }

```

출력 결과:

```

One
Two
Three

```

이 현상을 fall-through라고 한다. 의도적으로 사용할 수도 있지만, 초반에는 각 case의 마지막에 break를 작성하는 것이 안전하다.

5.3. if--else와 switch 비교

상황	적합한 문법
값의 범위를 검사	if--else
여러 논리 조건을 결합	if--else
하나의 값과 여러 상수를 비교	switch
메뉴 번호 선택	switch

6. 최소 한 번 실행: do--while

while문은 조건을 먼저 검사한다. 따라서 처음부터 조건이 거짓이면 본문이 한 번도 실행되지 않는다.

```

1 int number = 10;
2
3 while (number < 5)
4 {
5     cout << number << endl;
6 }

```

반면 do--while문은 본문을 먼저 실행한 뒤 조건을 검사한다.

```

1 do
2 {
3     // - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 켜
4     - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 꿀
5 } while (condition);

```

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int number = 10;

```



```

8
9     do
10    {
11        cout << number << endl;
12    }
13    while (number < 5);
14
15    return 0;
16 }

```

조건은 거짓이지만 본문이 먼저 실행되므로 10이 한 번 출력된다.

6.1. while과 do--while 비교

반복문	조건 검사 시점	최소 실행 횟수
while	본문 실행 전	0회
do--while	본문 실행 후	1회

사용자 입력을 반드시 한 번 받아야 하는 메뉴나 게임에서는 do--while문이 자연스럽다.

핵심 포인트

do--while문의 마지막에는 반드시 세미콜론이 필요하다.

7. 반복 즉시 종료: break

반복문 내부의 break는 가장 가까운 반복문을 즉시 종료한다.

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int number;
8
9      while (true)
10     {
11         cout << "Number: ";
12         cin >> number;
13
14         if (number == 0)
15         {
16             break;
17         }
18
19         cout << "You entered " << number << endl;
20     }
21
22     cout << "Loop ended." << endl;
23
24     return 0;
25 }

```

0을 입력하면 break가 실행되어 반복문을 빠져나간다. 반복문 다음에 있는 Loop ended.가 출력되고 프로그램이 계속 진행된다.

핵심 포인트

`break`는 프로그램 전체를 종료하는 명령이 아니다. 현재 실행 중인 가장 가까운 반복문 또는 `switch`문만 종료한다.

8. 현재 반복 건너뛰기: `continue`

`continue`는 반복문 전체를 끝내지 않는다. 현재 반복의 남은 코드를 건너뛰고 다음 반복으로 이동한다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     for (int i = 1; i <= 10; i++)
8     {
9         if (i % 2 == 0)
10        {
11            continue;
12        }
13
14        cout << i << endl;
15    }
16
17    return 0;
18 }
```

짝수일 때 `continue`가 실행되므로 출력문을 건너뛴다. 결과적으로 홀수만 출력된다.

8.1. `break`와 `continue` 비교

명령	동작
<code>break</code>	반복문 전체를 즉시 종료
<code>continue</code>	현재 반복만 건너뛰고 다음 반복 진행

8.2. 입력값 검증

```

1 int number;
2
3 while (true)
4 {
5     cout << "Positive number: ";
6     cin >> number;
7
8     if (number <= 0)
9     {
10        cout << "Invalid input." << endl;
11        continue;
12    }
13
14    cout << "Accepted: " << number << endl;
15    break;
16 }
```

잘못된 입력은 `continue`로 다시 입력받고, 올바른 입력이면 정상 작업을 수행한 뒤 `break`로 반복을 종료한다.

9. 난수 생성

C++에서 기존 C 표준 라이브러리 방식으로 난수를 생성하려면 `<cstdlib>`와 `<ctime>` 헤더를 사용한다.

```
1 #include <cstdlib>
2 #include <ctime>
```

9.1. rand()

`rand()`는 0 이상 `RAND_MAX` 이하의 정수를 반환한다.

```
1 #include <iostream>
2 #include <cstdlib>
3
4 using namespace std;
5
6 int main()
7 {
8     cout << rand() << endl;
9     cout << rand() << endl;
10    cout << rand() << endl;
11
12    return 0;
13 }
```

하지만 seed를 설정하지 않으면 프로그램을 다시 실행할 때 같은 순서의 값이 반복될 수 있다.

9.2. srand()와 seed

`srand()`는 난수 생성기의 시작값인 seed를 설정한다.

```
1 srand(10);
```

같은 seed를 사용하면 같은 난수 순서가 만들어진다. 실행할 때마다 다른 순서를 얻기 위해 현재 시간을 seed로 사용한다.

```
1 srand(time(0));
```

`time(0)`은 현재 시간을 초 단위 값으로 반환한다. 프로그램 시작 시 한 번만 호출하면 된다.

```
1 #include <iostream>
2 #include <cstdlib>
3 #include <ctime>
4
5 using namespace std;
6
7 int main()
8 {
9     srand(time(0));
10
11    cout << rand() << endl;
12    cout << rand() << endl;
13    cout << rand() << endl;
14
15    return 0;
16 }
```

핵심 포인트

`srand(time(0))`은 반복문 안에서 계속 호출하지 않는다. 프로그램 시작 부분에서 한 번만 호출한다.

10. 원하는 범위의 난수

`rand()`의 결과에 나머지 연산을 적용하면 범위를 제한할 수 있다.

```
1 int number = rand() % 10;
```

`rand() % 10`의 가능한 결과는 0부터 9까지이다.
1부터 10까지 만들려면 1을 더한다.

```
1 int number = rand() % 10 + 1;
```

일반적으로 최소값 `min`, 최대값 `max` 사이의 정수는 다음과 같이 만든다.

```
1 int number = rand() % (max - min + 1) + min;
```

예를 들어 50부터 100까지의 난수는 다음과 같다.

```
1 int number = rand() % 51 + 50;
```

가능한 나머지는 0부터 50까지이고, 여기에 50을 더하므로 최종 범위는 50부터 100까지가 된다.

체크 질문

`rand() % N`은 0부터 `N - 1`까지의 값을 만든다. 따라서 1부터 `N`까지가 필요하다면 마지막에 1을 더한다.

11. 종합 실습: 숫자 맞추기 게임

컴퓨터가 1부터 100 사이의 정답을 하나 만들고, 사용자가 정답을 맞힐 때까지 숫자를 입력하는 프로그램을 작성한다.

프로그램은 다음 기능을 가져야 한다.

- 정답은 프로그램 실행 때마다 무작위로 결정한다.
- 입력값이 정답보다 크면 Too high!를 출력한다.
- 입력값이 정답보다 작으면 Too low!를 출력한다.
- 1부터 100 범위를 벗어난 입력은 시도 횟수에 포함하지 않는다.
- 정답을 맞히면 총 시도 횟수를 출력하고 종료한다.

11.1. 전체 코드

```
1 #include <iostream>
2 #include <cstdlib>
3 #include <ctime>
4
5 using namespace std;
6
7 int main()
8 {
9     srand(time(0));
```

```
10
11     int answer = rand() % 100 + 1;
12     int guess;
13     int attempts = 0;
14
15     cout << "Number Guessing Game" << endl;
16     cout << "Guess a number from 1 to 100." << endl;
17
18     do
19     {
20         cout << "Guess: ";
21         cin >> guess;
22
23         if (guess < 1 || guess > 100)
24         {
25             cout << "Enter a number from 1 to 100." << endl;
26             continue;
27         }
28
29         attempts++;
30
31         if (guess > answer)
32         {
33             cout << "Too high!" << endl;
34         }
35         else if (guess < answer)
36         {
37             cout << "Too low!" << endl;
38         }
39         else
40         {
41             cout << "Correct!" << endl;
42         }
43     }
44     while (guess != answer);
45
46     cout << "Attempts: " << attempts << endl;
47     cout << "Game Over." << endl;
48
49     return 0;
50 }
```

11.2. 실행 흐름 분석

정답이 68이라고 가정하자. 사용자가 50을 입력하면 `guess < answer`가 참이므로 Too low!가 출력된다. 이후 `guess != answer`가 참이므로 반복한다.

75를 입력하면 `guess > answer`가 참이므로 Too high!가 출력된다. 마지막으로 68을 입력하면 앞의 두 조건이 모두 거짓이고 마지막 `else`가 실행된다. 이후 반복 조건이 거짓이 되어 게임이 종료된다.

입력값이 1부터 100 사이가 아니면 `continue`가 실행된다. 따라서 정답 비교와 시도 횟수 증가는 건너뛰고 다음 입력으로 이동한다.

12. 실습 파일 구성

실습

Day 2 파일은 다음과 같이 관리한다.

- 01_if_else.cpp
- 02_else_if.cpp
- 03_nested_if.cpp
- 04_switch.cpp
- 05_do_while.cpp
- 06_break.cpp
- 07_continue.cpp
- 08_random.cpp
- 09_random_range.cpp
- practice/p01_number_guessing.cpp

13. 핵심 요약

1. if--else는 두 경우 중 하나를 선택한다.
2. else if는 여러 조건을 위에서부터 순서대로 검사한다.
3. 중첩 조건문은 앞 단계가 통과된 뒤 추가 조건을 검사할 때 사용한다.
4. switch는 하나의 값과 여러 상수를 비교할 때 적합하다.
5. switch의 break가 없으면 fall-through가 발생할 수 있다.
6. do--while은 본문을 먼저 실행하므로 최소 한 번 실행된다.
7. break는 가장 가까운 반복문 또는 switch를 종료한다.
8. continue는 현재 반복의 남은 코드를 건너뛴다.
9. srand(time(0))은 프로그램 시작 시 한 번만 호출한다.
10. rand() % (max - min + 1) + min으로 지정 범위의 정수를 만들 수 있다.